

JAVA Programming

Lecture 3: Operators in JAVA



Department of Computer Science and Engineering
National Institute of Technology, Srinagar, Jammu and Kashmir
February 25, 2026

Identify Errors in the Given Program

```
class PracticeQuestions
{
    public static void main(int args[])
    {
        char c='k'; //line 1
        c=k; //line 2
        char c1="k"; //line 3
        char c2='9'; //line 4
        byte b=128; //line 5
        long l=999999999999; //line 6
        bool ready=False; //line 7
        float f=2.3; //line 8
    }
}
```

What will be the output?

```
int x=1_00_000_00_00;
```

```
System.out.println(x)
```

//output?

Output will be a normal number excluding all the underscores.

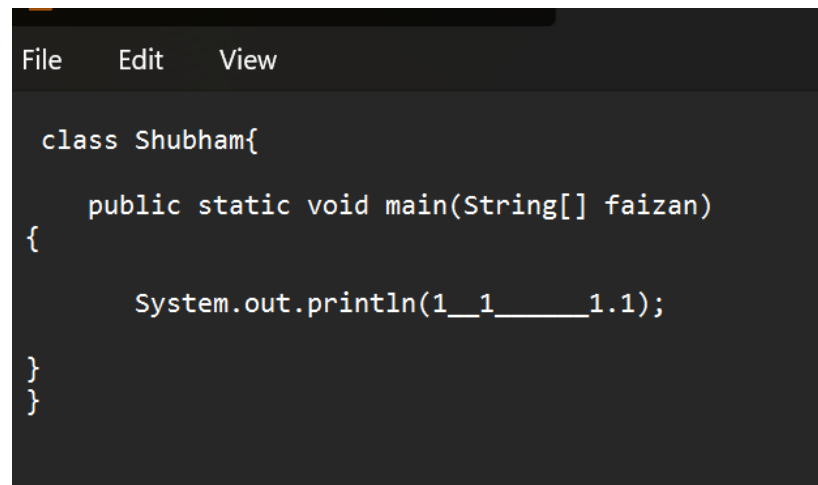
```
int x=1_00_000_00_00;
```

```
System.out.println(x)
```

Output will be a normal number excluding all the underscores.

The underscores in the NUMERIC Literals are used for better readability and do not affect the actual value.

This can be used for counting the number of zeros in the given literal.



```
File Edit View

class Shubham{
    public static void main(String[] faizan)
    {
        System.out.println(1__1_____1.1);
    }
}
```

What will be the output?

```
double f=54;
```

```
System.out.println(f)
```

```
//output?
```

What will be the output?

```
double f=54;  
System.out.println(f)
```

The program will compile successfully with no errors.

Output will be 54.0

The value 54 which is of int type will be converted to double

- In Java, everything goes in Class only. The source code (actual java code) is compiled and converted to the bytecode (**.class file**).
- Running a Java program means to tell the **JVM** to load the **“.class”** file and start executing the code in the **main()** method.
- **Main** method is where your program starts running.
- No matter how big your program is in term of number of classes, there will be **one main() method** in them to execute the program.
- Inside the main() method, you write the code.
- If you have multiple classes with main methods, and you want to execute a specific one, you need to specify the class name on the command line when running the Java program

Whitespace doesn't matter in Java

```
int    x    =    3;
```

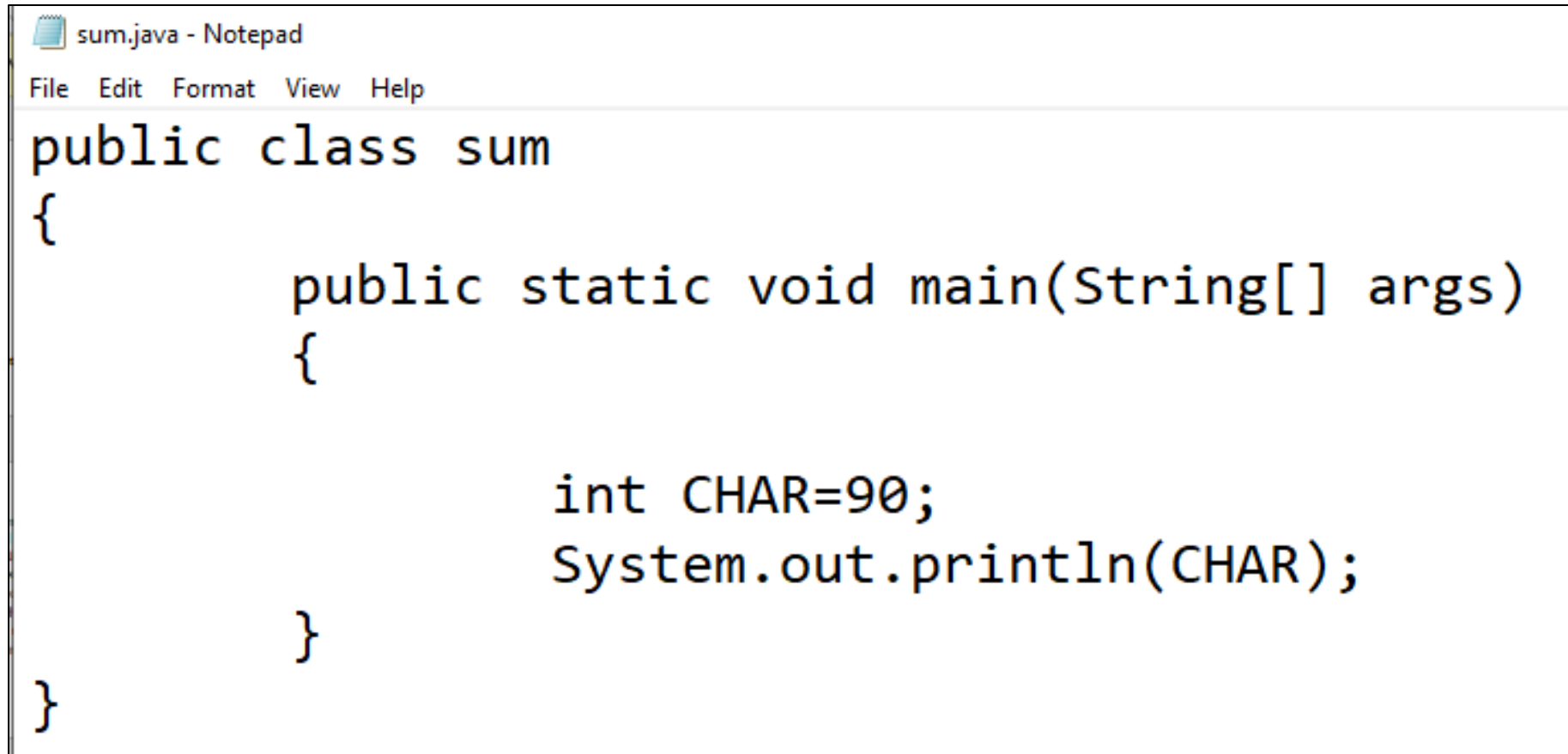
Is same as

```
int x=3;
```


Keywords in JAVA

- Keywords are the reserved words whose meaning is already defined in the JAVA Compiler.
- We can't use keywords for variable and other personal use.

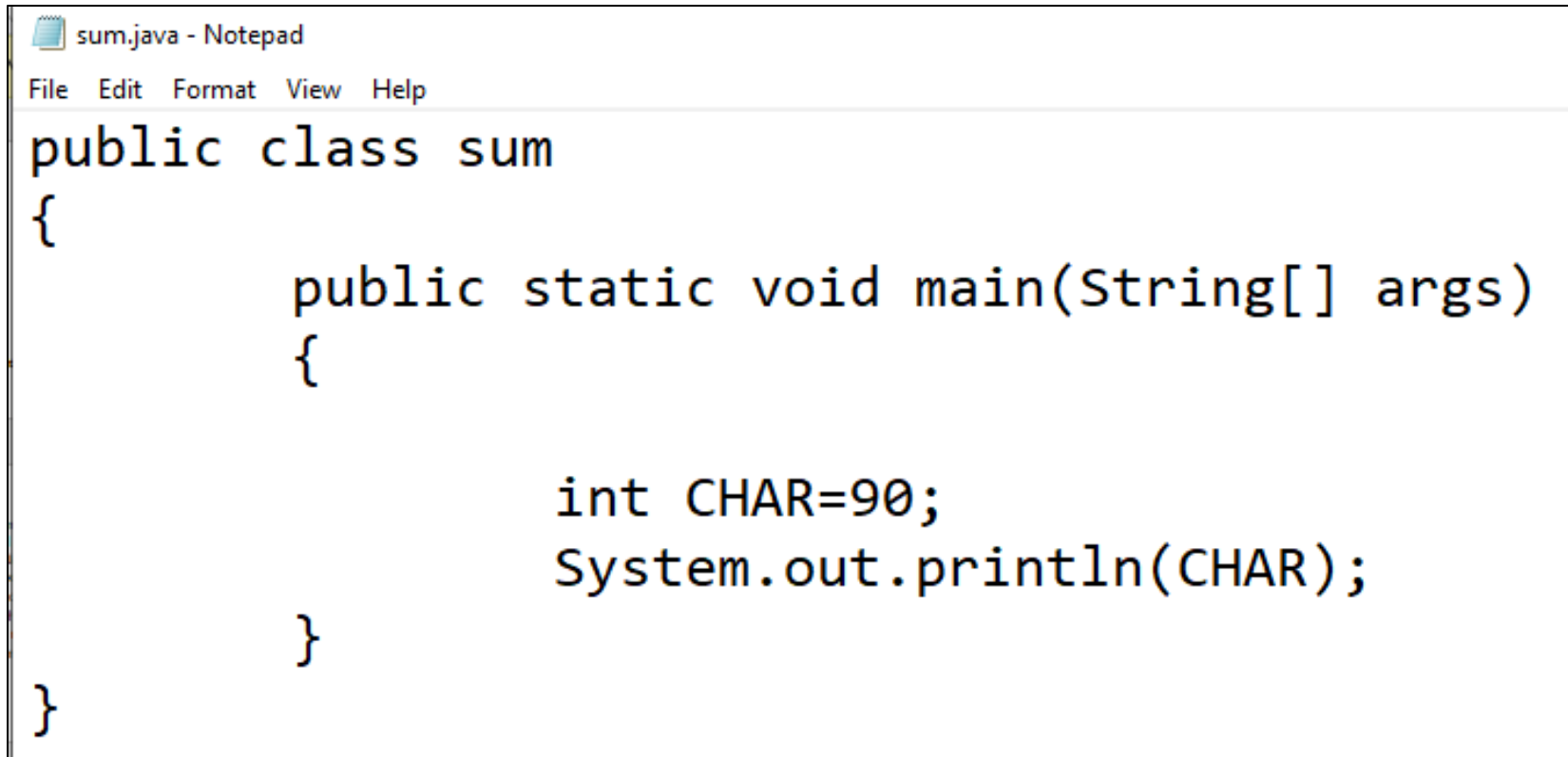
Output?



```
sum.java - Notepad
File Edit Format View Help
public class sum
{
    public static void main(String[] args)
    {
        int CHAR=90;
        System.out.println(CHAR);
    }
}
```

Keywords are case-sensitive.

- This code will execute successfully with no errors because **JAVA** has no keyword **“CHAR”**



```
sum.java - Notepad
File Edit Format View Help
public class sum
{
    public static void main(String[] args)
    {
        int CHAR=90;
        System.out.println(CHAR);
    }
}
```

Keywords in JAVA

byte	else	extends	import	Switch
Short	for	implements	class	Case
int	do	final	interface	Const *
long	while	finally	new	goto *
float	break	try	native	Strictfp **
double	continue	catch	instanceof	enum ****
void	default	throw	package	assert ***
char	private	throws	return	abstract
boolean	protected	static	this	transient
if	public	volatile	Super	Synchronized

Keywords in JAVA

- **null, true, false** are also keywords which can be used as literals only.

Operators in JAVA

- Operators are used to perform operations on variables and values.

```
int x = 100 + 50;
```

In the example above, we use the **+** operator to add together two values.

(Here, **100** and **50** are **operand** and **+** is the **operator** and **150** is the **result**)

Although the **+** operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or a variable and another variable:

```
int sum1 = 100 + 50;    // 150 (100 + 50)
```

```
int sum2 = sum1 + 250;  // 400 (150 + 250)
```

```
int sum3 = sum2 + sum1; // 800 (400 + 400)
```

Types of Operators in Java

- Arithmetic Operators: `[+, -, *, /, %, ++, --]`
- Assignment Operators: `[=, +=]`
- Comparison Operators: `[==, >=, <=]`
- Logical Operators: `[&&, ||, !]`
- Bitwise Operators: `[&, |]` (operates bitwise)

Arithmetic Operators

- Arithmetic operators are used to perform common mathematical operations.
- Arithmetic Operators: `[+, -, *, /, %, ++, --]`
 - `++` increments value of a variable by 1
 - `--` decrements value of a variable by 1
- Arithmetic operators cannot work with Booleans.
- `%` operator can work on floats and doubles.

% operator can work on floats and doubles.

```
System.out.println(12.5 % 3.5);
```

//here both the operands are of type *Double* and *result* will also be in *double* as the *default* data-type of fractional values are *double* in java.

```
System.out.println(12.5f % 3.5f)
```

//here both the *operands* are of type *float* as explicitly defined, and the *output* of the operation will also be *float*.

```
System.out.println(12.5f % 3.5d)
```

//here one operand is *float* and other is *double*, so output will be *Double* (output will be of bigger datatype)

What will be the Output?

```
System.out.println(10 % 2.5);
```

// It is possible to perform arithmetic operations between compatible datatypes.

//here **10** is of type *int* by default and **2.5** will be of type *double*.

Output will be 0 or 0.0 ?

What will be the Output?

```
System.out.println(1 * 2.5);
```

// It is possible to perform arithmetic operations between compatible datatypes.

//here **1** is of type *int* by default and **2.5** will be of type *double*.

Output will be 2 or 2.5 ?

Output?

sum.java - Notepad

File Edit Format View Help

```
public class sum
{
    public static void main(String[] args)
    {
        int a=10;
        int b=20;
        System.out.println(a++);
        System.out.println(a++);
        System.out.println(++a);
        System.out.println(++b);
        System.out.println(b++);
    }
}
```

Output?

```
sum.java - Notepad
File Edit Format View Help

public class sum
{
    public static void main(String[] args)
    {
        int a=10;
        int b=20;
        System.out.println(a++);
        System.out.println(a++);
        System.out.println(++a);
        System.out.println(++b);
        System.out.println(b++);
    }
}
```

Select C:\WINDOWS\system32\CMD.exe

```
G:\nit sri\Spring 2022\JAVA Programming 6th Sem CSE\code>javac sum.java
```

```
G:\nit sri\Spring 2022\JAVA Programming 6th Sem CSE\code>java sum
```

```
10
11
13
21
21
```

```
G:\nit sri\Spring 2022\JAVA Programming 6th Sem CSE\code>_
```

Output?

```
class P8
{
    public static void main(String args[])
    {
        int x = 5;
        System.out.println(x++ + ++x);
    }
}
```

Can we do this? Output

sum.java - Notepad

File Edit Format View Help

```
public class sum
{
    public static void main(String[] args)
    {
        int a=10;
        System.out.println(2++);
    }
}
```

No, Because increment operator works on variables and not on constants

```
sum.java - Notepad
File Edit Format View Help
public class sum
{
    public static void main(String[] args)
    {
        int a=10;
        System.out.println(2++);
    }
}
```

```
G:\nit sri\Spring 2022\JAVA Programming 6th Sem CSE\code>javac sum.java
sum.java:7: error: unexpected type
        System.out.println(2++);
                           ^
    required: variable
    found:    value
1 error
```


What will be the output?

```
char c='a';
```

```
c++;
```

// ++ is the increment operator

```
System.out.println(c);
```

Answer:

```
char c='a';
```

```
c++;
```

// ++ is the increment operator

```
System.out.println(c);
```

Output will be: **b**

Increment operator works on char datatypes also.

char is treated as an integer type

Range: 0 to 65535

It supports arithmetic operations

Increment Operators

```
int operand = 1;
```

```
++operand;
```

```
int number = ++operand;
```

```
// operand = 2
```

```
// operand = 3, number = 3
```

```
System.out.println('a'+'b');
```

% works with decimal numbers also

 *sum.java - Notepad

File Edit Format View Help

```
public class sum
{
    public static void main(String[] args)
    {
        int a=10;
        System.out.println(a%2.5);

    }
}
```

Java Assignment Operators

Assignment operators are used to assign values to variables.

Assignment Operators: [=, +=, -=, *=, /=, %=]

```
int x = 20;
```

The addition assignment operator (**+=**) adds a value to a variable:

```
int x = 10;
```

<code>x += 5;</code>	<code>x=x+5</code>	equivalent to <code>x = x + 5</code>
----------------------	--------------------	--------------------------------------

<code>x *= 3</code>	<code>x=x*3</code>	equivalent to <code>x = x * 3</code>
---------------------	--------------------	--------------------------------------

Comparison Operators

- Comparison operators are used to compare two values
- Comparison Operators: [==, >=, <=, >, <]

Bitwise Operators

- Bitwise operators in Java allow you to perform operations at the **bit level on integers (byte, short, int, and long) and char**
- These operators work directly on the binary representation of numbers.

Bitwise AND

&

Performs bitwise **AND** between two bits.

Bitwise OR

|

Performs bitwise **OR** between two bits

Bitwise XOR

^

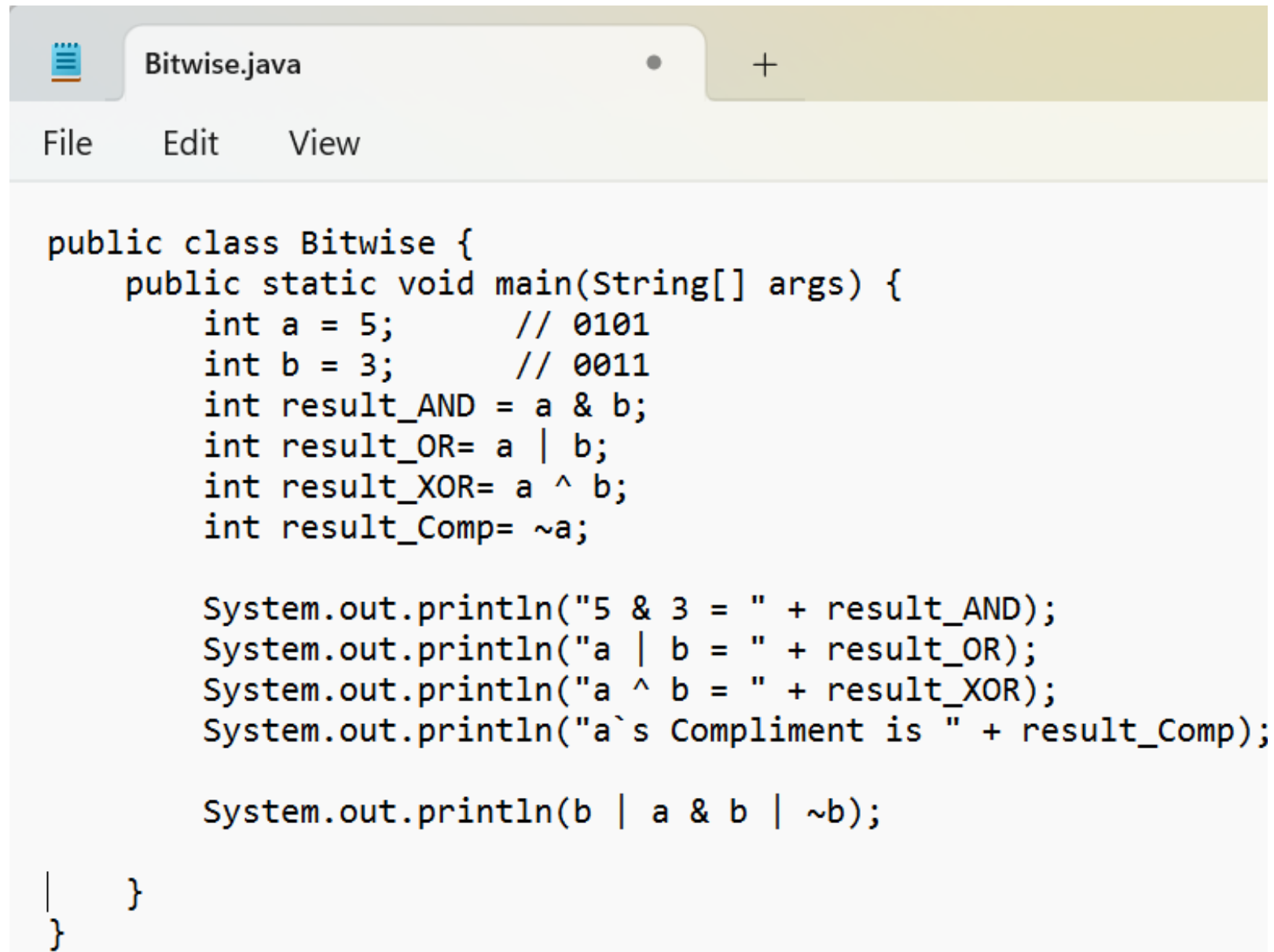
Performs bitwise **XOR** between two bits. (Different bits → 1)

Bitwise Complement

~

Inverts all bits (one's complement).

Output?



```
public class Bitwise {  
    public static void main(String[] args) {  
        int a = 5;        // 0101  
        int b = 3;        // 0011  
        int result_AND = a & b;  
        int result_OR = a | b;  
        int result_XOR = a ^ b;  
        int result_Comp = ~a;  
  
        System.out.println("5 & 3 = " + result_AND);  
        System.out.println("a | b = " + result_OR);  
        System.out.println("a ^ b = " + result_XOR);  
        System.out.println("a's Complement is " + result_Comp);  
  
        System.out.println(b | a & b | ~b);  
    }  
}
```


How to calculate $\sim N$ (Bitwise NOT)?

- The bitwise NOT ($\sim$) operator flips all bits of a number, including the sign bit
- Java uses **two's complement** representation for integers.
- In two's complement, numbers are stored in 32-bit signed representation for int. The leftmost bit (MSB) represents the sign (0 for positive, 1 for negative).

Steps:

1. *Convert N to its 32-bit binary representation.*
2. *Flip all the bits (0 becomes 1, 1 becomes 0).*
3. *Convert the result back to decimal using two's complement.*
 - *If the MSB is 1, the number is negative.*
 - *To get the decimal value of a negative number, invert all bits and add 1.*

OR

For any integer N :

$$\sim N = -(N+1)$$

Questions

1. Do we have to put a **main()** method in every class we write?
2. Why does everything have to be in a class in JAVA?
3. Write a JAVA Program to calculate the area of a rectangle.
4. Write a Program in JAVA to print your names and enrollment nos.
5. Program to calculate the remainder of dividing two numbers and print the output as "42 divided by 2 gives remainder 0"